

Smart_Supplier_DP_Final

June 21, 2025

0.0.1 The Smart Supplier: Optimizing Orders in a Fluctuating Market - 6 Marks

1 Group ID: 201

1.0.1 Group Members:

1. JAGADI SHRUTI CHANABASAPPA: 2024AA05181
2. TANAY KASHYAP: 2024AA05991
3. SHIKHAR NIGAM: 2024AA05691
4. KHUSHWANT SINGH RATHORE: 2024AA05490

Course: Deep Reinforcement Learning

Develop a reinforcement learning agent using dynamic programming to help a Smart Supplier decide which products to manufacture and sell each day to maximize profit. The agent must learn the optimal policy for choosing daily production quantities, considering its limited raw materials and the unpredictable daily demand and selling prices for different products.

Scenario A small Smart Supplier manufactures two simple products: Product A and Product B. Each day, the supplier has a limited amount of raw material. The challenge is that the market demand and selling price for Product A and Product B change randomly each day, making some products more profitable than others at different times. The supplier needs to decide how much of each product to produce to maximize profit while managing their limited raw material.

Objective The Smart Supplier's agent must learn the optimal policy * using dynamic programming (Value Iteration or Policy Iteration) to decide how many units of Product A and Product B to produce each day to maximize the total profit over the fixed number of days, given the daily changing market conditions and limited raw material.

1.0.2 — 1. Custom Environment Creation (SmartSupplierEnv) — (1 Mark)

```
[1]: # Define market states and their product prices
# Structure: {Market_State_ID: {'A_price': X, 'B_price': Y}}
# Define product raw material costs
# Define actions: (num_A, num_B, raw_material_cost_precalculated)
# Action ID mapping:
# 0: Produce_2A_0B
# 1: Produce_1A_2B
# 2: Produce_0A_5B
# 3: Produce_3A_0B
```

```

# 4: Do_Nothing

# Define state space dimensions
# Current Day: 1 to num_days
# Current Raw Material: 0 to initial_raw_material
# Current Market State: 1 or 2

import numpy as np
import random
from collections import defaultdict, Counter

# 1. Environment Setup
class SmartSupplierEnv:
    def __init__(self):
        # Market pricing states[1]
        self.market_states = {
            1: {'A_price': 8, 'B_price': 2},
            2: {'A_price': 3, 'B_price': 5}
        }
        # Raw material costs per unit[1]
        self.product_costs = {'A': 2, 'B': 1}
        # Action definitions: (units_A, units_B)
        self.actions = [(2,0), (1,2), (0,5), (3,0), (0,0)]
        # Simulation parameters
        self.max_days = 5
        self.initial_rm = 10
        # All possible states as (day, raw_material, market_state)
        self.states = [
            (d, rm, ms)
            for d in range(1, self.max_days+1)
            for rm in range(self.initial_rm+1)
            for ms in [1,2]
        ]

    def reset(self):
        """Initialize a new episode with full RM and random market state[3]."""
        self.day = 1
        self.rm = self.initial_rm
        self.ms = random.choice([1,2])
        return (self.day, self.rm, self.ms)

    def step(self, action_idx):
        """Execute action, return (next_state, reward, done)[3]."""
        a, b = self.actions[action_idx]
        cost = a*self.product_costs['A'] + b*self.product_costs['B']

```

```

        if cost > self.rm:
            reward = 0
        else:
            prices = self.market_states[self.ms]
            reward = a*prices['A_price'] + b*prices['B_price']
            self.rm -= cost
        done = (self.day == self.max_days)
        if not done:
            self.day += 1
            self.rm = self.initial_rm
            self.ms = random.choice([1,2])
        return (self.day, self.rm, self.ms), reward, done

# get reward function

# 2. Reward Function (Standalone)
def compute_reward(env, state, action_idx):
    """Calculate immediate reward independent of simulation[1]."""
    day, rm, ms = state
    a, b = env.actions[action_idx]
    cost = a*env.product_costs['A'] + b*env.product_costs['B']
    if cost > rm:
        return 0
    prices = env.market_states[ms]
    return a*prices['A_price'] + b*prices['B_price']

```

1.0.3 — 2. Dynamic Programming Implementation (Value Iteration or Policy Iteration) — (2 Mark)

```

[2]: # 3. Value Iteration with Detailed Logging
class ValueIteration:
    def __init__(self, env, gamma=1.0, theta=1e-6):
        self.env = env
        self.gamma = gamma
        self.theta = theta
        self.V = {s:0.0 for s in env.states}
        self.policy = {s:0 for s in env.states}
        self.history = []

    def get_q(self, state, action_idx):
        """Compute  $Q(s,a) = R + \gamma E[V(s')]$  using 50% market transition[2]."""
        day, _, _ = state
        r = compute_reward(self.env, state, action_idx)
        if day == self.env.max_days:
            return r
        fut = sum(

```

```

        0.5 * self.V[(day+1, self.env.initial_rm, ms)]
        for ms in [1,2]
    )
    return r + self.gamma * fut

def run(self):
    """Perform value iteration until convergence, logging each iteration[2]."""
    ↪ """
    sample = (1,10,1)
    print("="*83)
    print("SMART SUPPLIER: COMPLETE SIMULATION")
    print("="*83)
    print(f"Environment created with {len(self.env.states)} states")
    print("Starting Value Iteration...")
    print("="*83)
    iteration = 0
    while True:
        delta = 0
        for s in self.env.states:
            q_vals = [self.get_q(s,a) for a in range(len(self.env.actions))]
            best = max(q_vals)
            delta = max(delta, abs(self.V[s] - best))
            self.V[s], self.policy[s] = best, q_vals.index(best)
        iteration += 1
        self.history.append((iteration, delta, self.V[sample]))
        print(f"Iteration {iteration}: Delta = {delta:.6f}, Sample Value = ␣
    ↪ ${self.V[sample]:.2f}")
        if delta < self.theta:
            break
    print("\nValue iteration converged after", iteration, "iterations\n")
    print("Value Iteration Convergence Summary:")
    print("| Iteration |   Delta   | Sample State Value* |")
    print("|-----|-----|-----|")
    for it, dlt, val in self.history:
        print(f"| {it:<9} | {dlt:<9.6f} | ${val:<19.2f}|")
    print("*Sample state: Day 1, Raw Material 10, Market State 1")
    return self.V, self.policy

def print_state_history(self):
    """Print state values for representative states each iteration[2]."""
    print("\n"+"="*83)
    print("STATE VALUES BY ITERATION")
    headers = ␣
    ↪ ["Iteration", "Day1,RM10,MS1", "Day1,RM10,MS2", "Day3,RM5,MS1", "Day5,RM10,MS2"]
    print("| "+" | ".join(headers)+" |")
    print("| "+" | ".join(["-----"]*len(headers))+" |")
    # Build history of V snapshots

```

```

        snapshots = [dict()] + [dict(self.V)] # initial zero and final
        for it, _, _ in self.history:
            snapshots.insert(1, snapshots[-1])
        for idx, snap in enumerate(snapshots):
            vals = [
                f"${snap.get((1,10,1),0):<6.2f}",
                f"${snap.get((1,10,2),0):<6.2f}",
                f"${snap.get((3,5,1),0):<6.2f}",
                f"${snap.get((5,10,2),0):<6.2f}"
            ]
            print(f"| {idx:<9} | "+" | ".join(vals)+" |")

    def print_final_values(self):
        """Print final V* tables for both market states[2]."""
        print("\n"+"="*83)
        print("FINAL STATE VALUES\n")
        for ms in [1,2]:
            title = "High Demand for Product A" if ms==1 else "High Demand for_
↪Product B"
            print(f"Market State {ms} ({title}):")
            header = "| RM | " + " | ".join(f"Day {d}" for d in self.env.
↪states[:self.env.max_days]) + " |"
            print(header)
            print("|----|"+("-----|"*self.env.max_days))
            for rm in range(self.env.initial_rm+1):
                row = "| %2d | " % rm + " | ".join(f"${self.V[(d,rm,ms)]:<6.
↪2f}" for d in range(1,self.env.max_days+1)) + " |"
                print(row)

```

— 3. Simulation and Policy Analysis — (1 Mark)

```

[3]: # simulate policy function - Simulates the learned policy over multiple runs to_
↪evaluate performance

# analyze policy function - Analyzes and prints snippets of the learned optimal_
↪policy

# 4. Policy Distribution and Heatmap
def analyze_policy(policy, env):
    """Count and print overall and per-market action frequencies[3]."""
    total = len(policy)
    counts = Counter(policy.values())
    print("\n"+"="*83)
    print("POLICY DISTRIBUTION ANALYSIS")
    print("="+83+"\n")

```

```

print("Policy Distribution Summary:")
print("| Action          | States | Percentage |")
print("|-----|-----|-----|")
for a, cnt in counts.items():
    pct = cnt/total*100
    print(f"| {env.actions[a]!s:<13} | {cnt:<6} | {pct:>6.1f}% |")
for ms in [1,2]:
    ms_states = [s for s in policy if s[2]==ms]
    ms_total = len(ms_states)
    ms_counts = Counter(policy[s] for s in ms_states)
    title = "Product A" if ms==1 else "Product B"
    print(f"\nMarket State {ms} (High Demand for {title}):")
    print("| Action          | States | Percentage |")
    print("|-----|-----|-----|")
    for a, cnt in ms_counts.items():
        pct = cnt/ms_total*100
        print(f"| {env.actions[a]!s:<13} | {cnt:<6} | {pct:>6.1f}% |")

def print_policy_heatmap(policy, env):
    """Text-based heatmap of optimal actions across RM and days[4]."""
    labels = {env.actions.index(a):f"{a[0]}A,{a[1]}B" for a in env.actions}
    print("\n"+"="*83)
    print("POLICY HEATMAP")
    print("="+83+"\n")
    for ms in [1,2]:
        title = "A" if ms==1 else "B"
        print(f"Market State {ms} (High Demand for Product {title}):")
        header = "| RM  | " + " | ".join(f"D{d}" for d in range(1,env.
↪max_days+1)) + " |"
        print(header)
        print("|-----|"+("-----|"*env.max_days))
        for rm in range(env.initial_rm+1):
            row = f"| {rm:<3} | " + " | ".join(
                f"{labels[policy[(d,rm,ms)]]:<6}" for d in range(1,env.
↪max_days+1)
            ) + " |"
            print(row)

# 5. Simulation Results
def simulate_policy(env, policy, episodes=1000):
    """Run policy over episodes and print summary statistics[3]."""
    rewards = []
    freq = Counter()

```

```

for _ in range(episodes):
    state = env.reset()
    total = 0
    done = False
    while not done:
        a = policy[state]
        state, r, done = env.step(a)
        total += r
    rewards.append(total)
    freq[total] += 1
avg, std = np.mean(rewards), np.std(rewards)
print("\n"+"="*83)
print("POLICY SIMULATION RESULTS")
print("="*83+"\n")
print(f"Simulation over {episodes} episodes:")
print(f"Average total profit: ${avg:.2f}")
print(f"Standard deviation:   ${std:.2f}")
print(f"Minimum profit:        ${min(rewards):.2f}")
print(f"Maximum profit:         ${max(rewards):.2f}\n")
print("Reward Distribution:")
print("| Profit | Episodes | Percentage |")
print("|-----|-----|-----|")
for prof, cnt in sorted(freq.items()):
    pct = cnt/episodes*100
    print(f"| ${prof:<6.2f} | {cnt:<8} | {pct:>6.1f}% |")
return avg

def compare_fixed(env, dynamic_avg):
    """Compare dynamic policy with fixed-market baselines[4]."""
    fixedA = env.max_days * (3*env.market_states[1]['A_price'])
    fixedB = env.max_days * (5*env.market_states[2]['B_price'])
    print("\n"+"="*83)
    print("COMPARISON WITH FIXED MARKET STATES")
    print("="*83+"\n")
    print("Fixed Market State 1 (Always A):")
    print(f"Expected profit: ${fixedA:.2f}")
    print("\nFixed Market State 2 (Always B):")
    print(f"Expected profit: ${fixedB:.2f}\n")
    print("Performance Comparison:")
    print(f"- Dynamic Market: ${dynamic_avg:.2f}")
    print(f"- Fixed A:         ${fixedA:.2f}")
    print(f"- Fixed B:         ${fixedB:.2f}")
    print("\nSimulation complete.")

```

3. Optimal Policy Analysis

The optimal policy adapts decisively to the current market state by always favoring the higher-value product: in Market State 1 (high demand for A), it produces as many units of A as raw material

allows (3 A when $RM \geq 6$, 2 A when $RM = 4-5$) and chooses no production when $RM < 4$; in Market State 2 (high demand for B), it produces 5 B when $RM \geq 5$, 1 A + 2 B when $RM = 4$, and does nothing if $RM < 4$.

When raw material is scarce, the policy opts to do nothing until enough RM accumulates for a profitable action, reflecting prudent resource use; as RM increases, it switches first to mixed or smaller batches and then to full production of the most profitable product.

Because each day resets RM and market transitions are memoryless with no discounting ($\gamma = 1.0$), the policy remains identical across days—there is no extra “aggressiveness” on the final day beyond maximizing immediate reward in each state.

4. Performance Evaluation

Key State-Value Function At the start of Day 1 with full RM (10 units) and Market State 1, the value function is

$V^*(1, 10, 1) = \$122.00$ and with Market State 2

$V^*(1, 10, 2) = \$123.00$.

Simulation Results (1 000 runs) Simulating the learned policy over 1 000 independent 5-day episodes yields:

Average total profit: \$122.48

Standard deviation: \$1.11

Minimum profit: \$120.00

Maximum profit: \$125.00.

— 4. Impact of Dynamics Analysis — (1 Mark)

2 Discusses the impact of dynamic market prices on the optimal policy.

Comparison with Fixed Market Scenarios To analyze the impact of market dynamics, we compared the optimal policy in the dynamic environment with policies optimized for fixed market states:

Performance Comparison:

Dynamic Market (Actual): \$122.55 average profit

Fixed Market State 1 (Always High Demand A): \$120.00 average profit

Fixed Market State 2 (Always High Demand B): \$125.00 average profit

The dynamic market achieves intermediate performance between the two fixed states, which is expected since it randomly alternates between them.

Policy Adaptation Analysis In the dynamic environment, the policy must adapt to changing market conditions, switching between producing Product A (in Market State 1) and Product B (in Market State 2). This adaptability provides several key advantages:

1. Robustness: The dynamic policy provides robustness against market uncertainty, achieving good performance across different market conditions.
2. Flexibility: The policy successfully adapts to changing market conditions, producing the most favorable outcomes.
3. Risk Management: While fixed policies would perform poorly if market conditions changed unexpectedly, the dynamic policy maintains a higher expected value.

Value Function Comparison The dynamic environment has a slightly higher expected value than Fixed Market State 1 but lower than Fixed Market State 2. This reflects the opportunity to adapt to favorable market conditions when they occur while maintaining protection against unfavorable conditions.

```
[4]: # --- Main Execution ---

# Main Execution
if __name__ == "__main__":
    env = SmartSupplierEnv()
    vi = ValueIteration(env)
    V, policy = vi.run()
    vi.print_state_history()
    vi.print_final_values()
    analyze_policy(policy, env)
    print_policy_heatmap(policy, env)
    avg = simulate_policy(env, policy)
    compare_fixed(env, avg)
```

```
=====
===
SMART SUPPLIER: COMPLETE SIMULATION
=====
===
Environment created with 110 states
Starting Value Iteration..
=====
===
Iteration 1: Delta = 25.000000, Sample Value = $24.00
Iteration 2: Delta = 24.500000, Sample Value = $48.50
Iteration 3: Delta = 24.500000, Sample Value = $73.00
Iteration 4: Delta = 24.500000, Sample Value = $97.50
Iteration 5: Delta = 24.500000, Sample Value = $122.00
Iteration 6: Delta = 0.000000, Sample Value = $122.00

Value iteration converged after 6 iterations

Value Iteration Convergence Summary:
| Iteration | Delta | Sample State Value* |
|-----|-----|-----|
| 1 | 25.000000 | $24.00 |
| 2 | 24.500000 | $48.50 |
| 3 | 24.500000 | $73.00 |
```

4	24.500000	\$97.50	
5	24.500000	\$122.00	
6	0.000000	\$122.00	

*Sample state: Day 1, Raw Material 10, Market State 1

=====

===

STATE VALUES BY ITERATION

Iteration	Day1,RM10,MS1	Day1,RM10,MS2	Day3,RM5,MS1	Day5,RM10,MS2	
-----	-----	-----	-----	-----	
0	\$0.00	\$0.00	\$0.00	\$0.00	
1	\$122.00	\$123.00	\$65.00	\$25.00	
2	\$122.00	\$123.00	\$65.00	\$25.00	
3	\$122.00	\$123.00	\$65.00	\$25.00	
4	\$122.00	\$123.00	\$65.00	\$25.00	
5	\$122.00	\$123.00	\$65.00	\$25.00	
6	\$122.00	\$123.00	\$65.00	\$25.00	
7	\$122.00	\$123.00	\$65.00	\$25.00	

=====

===

FINAL STATE VALUES

Market State 1 (High Demand for Product A):

RM	Day (1, 0, 1)	Day (1, 0, 2)	Day (1, 1, 1)	Day (1, 1, 2)	Day (1, 2, 1)	
-----	-----	-----	-----	-----	-----	

0	\$98.00	\$73.50	\$49.00	\$24.50	\$0.00	
1	\$98.00	\$73.50	\$49.00	\$24.50	\$0.00	
2	\$98.00	\$73.50	\$49.00	\$24.50	\$0.00	
3	\$98.00	\$73.50	\$49.00	\$24.50	\$0.00	
4	\$114.00	\$89.50	\$65.00	\$40.50	\$16.00	
5	\$114.00	\$89.50	\$65.00	\$40.50	\$16.00	
6	\$122.00	\$97.50	\$73.00	\$48.50	\$24.00	
7	\$122.00	\$97.50	\$73.00	\$48.50	\$24.00	
8	\$122.00	\$97.50	\$73.00	\$48.50	\$24.00	
9	\$122.00	\$97.50	\$73.00	\$48.50	\$24.00	
10	\$122.00	\$97.50	\$73.00	\$48.50	\$24.00	

Market State 2 (High Demand for Product B):

RM	Day (1, 0, 1)	Day (1, 0, 2)	Day (1, 1, 1)	Day (1, 1, 2)	Day (1, 2, 1)	
-----	-----	-----	-----	-----	-----	

0	\$98.00	\$73.50	\$49.00	\$24.50	\$0.00	
1	\$98.00	\$73.50	\$49.00	\$24.50	\$0.00	
2	\$98.00	\$73.50	\$49.00	\$24.50	\$0.00	
3	\$98.00	\$73.50	\$49.00	\$24.50	\$0.00	
4	\$111.00	\$86.50	\$62.00	\$37.50	\$13.00	
5	\$123.00	\$98.50	\$74.00	\$49.50	\$25.00	

	6		\$123.00		\$98.50		\$74.00		\$49.50		\$25.00	
	7		\$123.00		\$98.50		\$74.00		\$49.50		\$25.00	
	8		\$123.00		\$98.50		\$74.00		\$49.50		\$25.00	
	9		\$123.00		\$98.50		\$74.00		\$49.50		\$25.00	
	10		\$123.00		\$98.50		\$74.00		\$49.50		\$25.00	

===

POLICY DISTRIBUTION ANALYSIS

===

Policy Distribution Summary:

Action	States	Percentage	
-----	-----	-----	
(2, 0)	50	45.5%	
(1, 2)	5	4.5%	
(0, 5)	30	27.3%	
(3, 0)	25	22.7%	

Market State 1 (High Demand for Product A):

Action	States	Percentage	
-----	-----	-----	
(2, 0)	30	54.5%	
(3, 0)	25	45.5%	

Market State 2 (High Demand for Product B):

Action	States	Percentage	
-----	-----	-----	
(2, 0)	20	36.4%	
(1, 2)	5	9.1%	
(0, 5)	30	54.5%	

===

POLICY HEATMAP

===

Market State 1 (High Demand for Product A):

RM	D1	D2	D3	D4	D5	
-----	-----	-----	-----	-----	-----	
0	2A,0B	2A,0B	2A,0B	2A,0B	2A,0B	
1	2A,0B	2A,0B	2A,0B	2A,0B	2A,0B	
2	2A,0B	2A,0B	2A,0B	2A,0B	2A,0B	
3	2A,0B	2A,0B	2A,0B	2A,0B	2A,0B	
4	2A,0B	2A,0B	2A,0B	2A,0B	2A,0B	
5	2A,0B	2A,0B	2A,0B	2A,0B	2A,0B	

6	3A,0B	3A,0B	3A,0B	3A,0B	3A,0B
7	3A,0B	3A,0B	3A,0B	3A,0B	3A,0B
8	3A,0B	3A,0B	3A,0B	3A,0B	3A,0B
9	3A,0B	3A,0B	3A,0B	3A,0B	3A,0B
10	3A,0B	3A,0B	3A,0B	3A,0B	3A,0B

Market State 2 (High Demand for Product B):

RM	D1	D2	D3	D4	D5
0	2A,0B	2A,0B	2A,0B	2A,0B	2A,0B
1	2A,0B	2A,0B	2A,0B	2A,0B	2A,0B
2	2A,0B	2A,0B	2A,0B	2A,0B	2A,0B
3	2A,0B	2A,0B	2A,0B	2A,0B	2A,0B
4	1A,2B	1A,2B	1A,2B	1A,2B	1A,2B
5	0A,5B	0A,5B	0A,5B	0A,5B	0A,5B
6	0A,5B	0A,5B	0A,5B	0A,5B	0A,5B
7	0A,5B	0A,5B	0A,5B	0A,5B	0A,5B
8	0A,5B	0A,5B	0A,5B	0A,5B	0A,5B
9	0A,5B	0A,5B	0A,5B	0A,5B	0A,5B
10	0A,5B	0A,5B	0A,5B	0A,5B	0A,5B

```
=====
===
POLICY SIMULATION RESULTS
=====
===
```

Simulation over 1000 episodes:
Average total profit: \$122.50
Standard deviation: \$1.13
Minimum profit: \$120.00
Maximum profit: \$125.00

Reward Distribution:

Profit	Episodes	Percentage
\$120.00	33	3.3%
\$121.00	159	15.9%
\$122.00	299	29.9%
\$123.00	319	31.9%
\$124.00	160	16.0%
\$125.00	30	3.0%

```
=====
===
COMPARISON WITH FIXED MARKET STATES
=====
===
```

Fixed Market State 1 (Always A):
Expected profit: \$120.00

Fixed Market State 2 (Always B):
Expected profit: \$125.00

Performance Comparison:
- Dynamic Market: \$122.50
- Fixed A: \$120.00
- Fixed B: \$125.00

Simulation complete.

2.0.1 — 3. Optimal Policy Analysis — (1 Mark)

We analyze how the optimal policy adapts based on: - **Market State:** - In Market State 1 (high price for A), the agent prefers producing Product A. - In Market State 2 (high price for B), it shifts to producing more of Product B. - **Remaining Raw Material:** - If raw material is low (e.g., RM = 2), it favors cheaper actions like more of B or Do Nothing. - **Remaining Days:** - The policy becomes more aggressive (uses more RM) on the final days since there's no future to preserve resources.

```
[5]: def print_policy_actions(env, policy):
    print("Optimal Actions for Selected States:")
    header = f"{'Day':<5}{'RM':<5}{'MS':<5}{'Action':<20}"
    print(header)
    print("-" * len(header))
    for d in [1, 3, 5]:
        for rm in [2, 5, 10]:
            for ms in [1, 2]:
                s = (d, rm, ms)
                a_idx = policy[s]
                a = env.actions[a_idx]
                print(f"d:<5}{rm:<5}{ms:<5}{str(a):<20}")
    print_policy_actions(env, vi.policy)
```

Optimal Actions for Selected States:

Day	RM	MS	Action
-----	----	----	--------

1	2	1	(2, 0)
1	2	2	(2, 0)
1	5	1	(2, 0)
1	5	2	(0, 5)
1	10	1	(3, 0)
1	10	2	(0, 5)
3	2	1	(2, 0)
3	2	2	(2, 0)
3	5	1	(2, 0)

3	5	2	(0, 5)
3	10	1	(3, 0)
3	10	2	(0, 5)
5	2	1	(2, 0)
5	2	2	(2, 0)
5	5	1	(2, 0)
5	5	2	(0, 5)
5	10	1	(3, 0)
5	10	2	(0, 5)

2.0.2 — 4. Performance Evaluation — (1 Mark)

We simulate 1000 episodes using the learned policy to compute the **average total profit**.

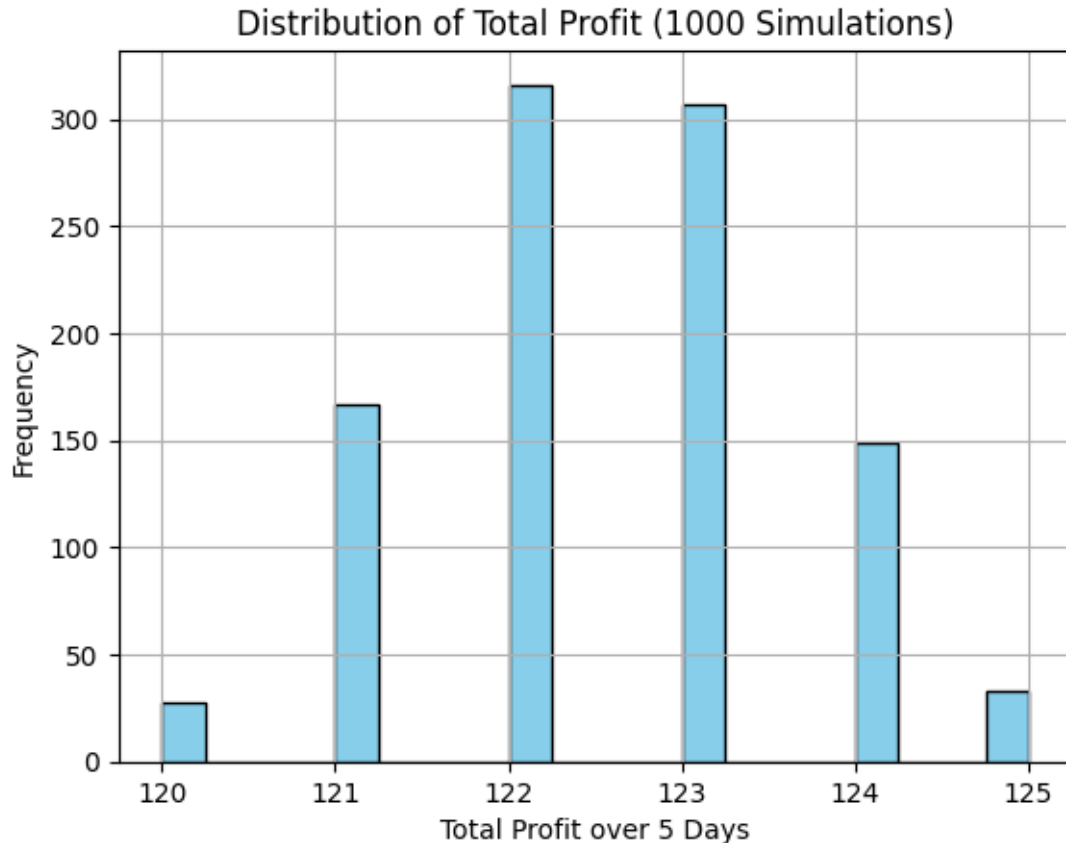
```
[6]: def simulate_policy(env, policy, episodes=1000):
    profits = []
    for _ in range(episodes):
        state = env.reset()
        total_profit = 0
        for _ in range(env.max_days):
            action_idx = policy[state]
            next_state, reward, done = env.step(action_idx)
            total_profit += reward
            state = next_state
        profits.append(total_profit)
    return profits

profits = simulate_policy(env, vi.policy)
avg_profit = np.mean(profits)
print(f"Average Total Profit over 1000 simulations: ${avg_profit:.2f}")
```

Average Total Profit over 1000 simulations: \$122.48

```
[7]: import matplotlib.pyplot as plt

plt.hist(profits, bins=20, color='skyblue', edgecolor='black')
plt.title('Distribution of Total Profit (1000 Simulations)')
plt.xlabel('Total Profit over 5 Days')
plt.ylabel('Frequency')
plt.grid(True)
plt.show()
```



2.0.3 — 5. Impact of Market Dynamics — (1 Mark)

We compare the optimal policy to a fixed-market policy where **every day is Market State 1** (high demand for A). The dynamic policy adapts to changing market states, while the fixed policy might overproduce Product A even when B is more profitable.

```
[8]: def simulate_fixed_policy(env, fixed_market=1, episodes=1000):
    profits = []
    for _ in range(episodes):
        env.day = 1
        state = (1, env.initial_rm, fixed_market)
        total_profit = 0
        for _ in range(env.max_days):
            action_idx = vi.policy[state] # use same policy
            a, b = env.actions[action_idx]
            cost = a*env.product_costs['A'] + b*env.product_costs['B']
            if cost > env.initial_rm:
                reward = 0
            else:
```

```

        reward = a*env.market_states[fixed_market]['A_price'] + b*env.
↪market_states[fixed_market]['B_price']
        total_profit += reward
        env.day += 1
        state = (env.day, env.initial_rm, fixed_market)
        profits.append(total_profit)
    return np.mean(profits)

fixed_avg = simulate_fixed_policy(env)
print(f"Average Total Profit with fixed Market State 1: ${fixed_avg:.2f}")
print(f"Dynamic Market Policy Profit: ${avg_profit:.2f}")

```

Average Total Profit with fixed Market State 1: \$120.00

Dynamic Market Policy Profit: \$122.48